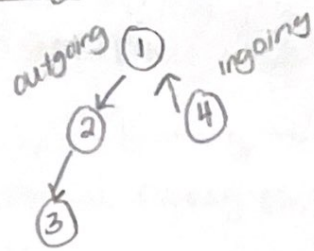
Erin Berry Q1 HW9

1) G is directed. After running DFS on G, the vertex v of G ended up in a DFS tree containing only v, even though v has both incoming and outgoing edges in G.

directed Graph G



Lets say we run DFS(G,1) where 1 vertex serves as our v vertex in our problem. First DFS will print out the right subtree of all outgoing vertices 1,2,3. Since this is Directed then we are only printing the vertices from 1 to 3 (lets assume our alg uses pre order output). Then, vertex 1 (or "v") is visited and will only print of 1 because it does NOT have any outgoing vertices it has Not visited, and thus vertex 1 ("v") ends up in a DFS tree containing only 1 ("v") deppite having both incoming and outgoing edges

Erin Bloz Q2 Homework 9

1) given an undirected connected graph $G=(V,E)$, develop an alg that determines whether G has a cycle. Alg TC $\simeq O(|V|+|E|)$

cycle: A path w. the same start and end vertices

Plan: DFS alg, visit each vertex once, if seen list is greater than 1 there is a cycle from our source vertex, else, theres NOT (source vertex does NOT have any children)

```
CheckCycle_DFS(G,S)
    //will Not keep track of time
    S.l = active   //set source vertex label = active, will only become seen if
                   //all reachable vertices have been discovered.
    counting = 1   //track how many vertices we visit
    for each v in V[S]   //adjacency list of s (all vertices that make an edge w.s)
        if v.l == Undiscovered   //undiscovered child
            counting += 1   //increment count
            v.Parent = S   //set source vertex as parent

            DFS(G,V)   //recursively call DFS on this vertex

    S.l = seen   //we discovered all reachable vertices
    if (counting > 1)
        return True   //We visited more than one vertex from our
                      //start vertex and end vertex which are
                      //both the same (source vertex)
    return False
```

① $O(|V|)$

② $O(|E|)$

TC: $O(|V|+|E|)$ as seen in circled parts 1 & 2, the for loop iterates $|V|$ times and TC of part 2 depends on $|E|$. Thus total TC $= O(|V|+|E|)$

Eem Bury

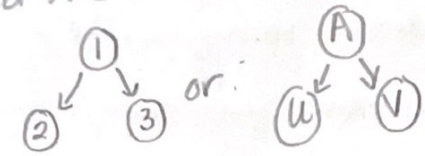
Q3 HW9

3a) Provide counterex: If a directed graph G contains a path from u to v, and if u.d < v.d (discovery time) in DFS(G) the v is a descendant of u in DFS forest

Note: Forest: Union of disjoint trees

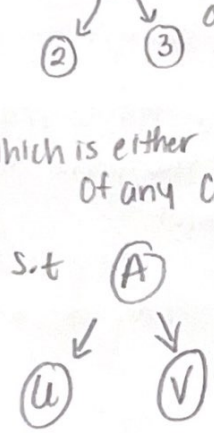
u was discovered first. WTS: u.d < v.d but v != descendant of u

directed graph G



descendant of u: any vertex which is either the child of u or is the descendant (child) of any children of u.

using directed graph G s.t



we see v is NOT a descendant of u. Rather, u and v are descendant of A.

Running DFS(G, A) s.t we use a pre order output, we see u.d < v.d because we visit our source vertex A, then u and then v. Thus, u is discovered first (u.d < v.d) and again, v is NOT a descendant of u. Δ

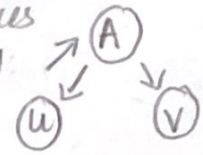
(we produce a forest since G is directed)

3b) If a directed graph G contains a path u to v, the any DFS(G) must result in v.d ≤ u.f

WTS: path u to v exists, v.d > u.f, discover u first

↳ walk w/o repeated vertices

Consider directed Graph G:



Here, if we apply DFS(G, A) and use labels + pre order output, as well as track time we will visit A-u-A-v where u is "seen" before even visiting v for the first time thus v.d > u.f! And a path exists Δ

Evan Blanz

Q4 HW9

4) Directed graph, 2 vertices u, v in graph, function P finds Shortest Path from u to v

consider directed graph G s.t each edge is colored red or blue. Given 2 vertices s, t in G we WTF shortest path from s to t which is "Color Separable" (i.e find red edges (can be 0) and then blue (can be none))

Develop an $O(|V| + |E| + T_P)$ - time alg to find shortest color separable path from s to t, where T_P denotes running time of P

→ Dont use BFS or explore graph yourself. Instead call function P

Justify TC. Hint: Consider the graph with only red edges of G, then only the blue edges. Combine these two graphs to construct new graph & solve

Problem: perhaps end up with a Forest

//consider graph with ONLY red edges: create copy w. only red edges

Color Separable

//assume redEdges and BlueEdges graphs have been computed (redEdges: only red edges in G, blueEdges: only blue edges in G)

// aux arrays redShortestPath + blueShortestPath store shorest Path of vertex v: A[v] Stores shortest Path from Source vertex s to v (only using red or blue edges respectively)

For each v in redEdges //redEdges is graph

$O(|R|V|)$ $O(|R|E|)$ { if (v.l == undiscovered)
{ redShortestpath[v] = P(s, v) $O(T_P)$ (starting → end)

$O(|B|V|)$ For each v in blueEdges

$O(|B|E|)$ b'cuz edges if (v.l = undiscovered)
blueshortest path[v] = P(v, t) //only using blue $O(T_P)$ (start where we last left off w/ red edge)

$O(|V|)$ For each v in G: //add the paths, they line up index wise: We can concatenate paths
//create another

red blue path = redshortest path[v] + blueshortest path[v]

return min(redblue path)

TC: $O(2|V|) + O(|E|) + O(T_P)$ ← TC of func. P

$O(|R| + |B| + V)$ ($|R| + |B| = |V|$) $O(\text{Red edges} + \text{blue edges}) = O(|E|)$